SOFTWARE REQUIREMENTS SPECIFICATION

Ethiopian Language Interpretation & Call Center Platform

Version: 1.0 — Draft for Review

Date: October 2026

React.js / TypeScript • Flutter Android • Django REST Framework • PostgreSQL

Proposed development budget reference: ETB 90,000

Organization administration is excluded from this SRS release

Document Control

Field	Details
Document title	Ethiopian Language Interpretation & Call Center Platform — SRS
Document type	Software Requirements Specification
Version	1.0 — Draft for Review
Date	October 2026
Primary audience	Project owner, developers, testers, interpreters, supervisors, pilot stakeholders
Backend / database	Django REST Framework / PostgreSQL
Mobile application	Flutter Android
Web frontend	React.js / TypeScript
Scope exclusion	Organization administration and organization self-service administration are excluded

Document status: Proposed baseline requiring stakeholder validation and approval before development.

Table of Contents

1. Introduction 6
- 1.1 Purpose 6
- 1.2 Scope 6
- 1.3 Goals and Objectives 6
- 1.4 Intended Readers 6
- 1.5 Definitions and Acronyms 6
- 1.6 Reference Documents and Status 7

2. Overall Description 7
- 2.1 Product Perspective 7
- 2.2 Operating and Funding Model 7
- 2.3 User Classes and Roles 7
- 2.4 Operating Environment 7
- 2.5 Design and Implementation Constraints 8
- 2.6 Assumptions and Dependencies 8
- 2.7 In-Scope Capabilities 8
- 2.8 Out of Scope 8

3. Functional Requirements 9
- 3.1 Authentication and Access Control 9
- 3.2 User and Interpreter Profiles 9
- 3.3 Language Catalog and Unknown-Language Handling 9
- 3.4 Request Creation and Validation 10
- 3.5 Matching and Dispatch 10
- 3.6 Scheduled Requests 10
- 3.7 Session Creation and Communication 11
- 3.8 Session Completion and Feedback 11
- 3.9 Supervisor / Coordinator Workspace 11
- 3.10 Platform Administration 11
- 3.11 Notifications 12
- 3.12 Operational Reports 12
- 3.13 Billing and Commercial Records — Scope Boundary 12
- 3.14 Error Handling and Recovery 12

4. External Interface Requirements 13
- 4.1 User Interface 13
- 4.2 Flutter Android Application 13
- 4.3 React Web Platform 13
- 4.4 Backend API 13
- 4.5 Database Interface 13
- 4.6 Real-Time Events and Media 14
- 4.7 External Services 14

5. Data Requirements and Proposed Data Model 14
- 5.1 Data Principles 14

5.2 Core Entities 14
5.3 Relationships 15
5.4 Data Validation and Integrity 15
5.5 Retention, Archival, and Deletion 15
5.6 Test Data and Seeding 15
6. Non-Functional Requirements 15
6.1 Performance 15
6.2 Availability and Reliability 15
6.3 Security 16
6.4 Privacy and Confidentiality 16
6.5 Usability and Accessibility 16
6.6 Maintainability 16
6.7 Scalability 16
6.8 Compatibility and Observability 16
7. Security, Safety, and Responsible Operation 17
7.1 Role-Based Access 17
7.2 High-Stakes Contexts 17
7.3 Unknown Language and Escalation 17
7.4 Recording and Consent 17
7.5 Audit and Incident Response 17
7.6 Abuse Prevention 17
8. Workflows and State Models 18
8.1 Immediate Request Workflow 18
8.2 Scheduled Request Workflow 18
8.3 Assignment Retry Workflow 18
8.4 Proposed Session States 18
8.5 Proposed Request States 18
8.6 Phone or Speakerphone Access 18
9. System Architecture and Technology 19
9.1 Logical Architecture 19
9.2 Backend Modules 19
9.3 Web Modules 19
9.4 Android Modules 19
9.5 Integration Design 19
9.6 Environments and Deployment 19
9.7 Technology Baseline 20
10. Testing and Acceptance 20
10.1 Test Levels 20
10.2 Essential Test Scenarios 20
10.3 Acceptance Criteria 20
10.4 Defect Severity 20
10.5 Pilot Readiness 21
11. Implementation Plan and Deliverables 21

11.1 Proposed Phases 21
11.2 Deliverables 21
11.3 Schedule 21
11.4 Change Control 21
11.5 Handover and Support 21
12. Risks, Dependencies, and Open Decisions 22
12.1 Risks 22
12.2 Mitigation 22
12.3 Decisions Required Before Development 22
12.4 External Dependencies 22
12.5 Budget Boundary 22
13. Appendices 23
Appendix A. Suggested API Domains 23
Appendix B. Initial Role Permission Summary 23
Appendix C. Requirement Priority 23
Appendix D. Approval Record 23

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, interface, data, security, operational, and quality requirements for the first release of the Ethiopian Language Interpretation & Call Center Platform. It is the baseline for design, development, testing, pilot preparation, and acceptance. It is intended for the project owner, developers, testers, interpreters, supervisors/coordinators, and pilot stakeholders.

1.2 Scope

The platform coordinates human interpretation between people who need language assistance and qualified interpreters. It supports request intake, language and qualification matching, interpreter availability, request acceptance and reassignment, audio/video or approved telephone-assisted sessions, completion records, feedback, operational monitoring, and aggregate usage reports. It is intended for any Ethiopian organization that requires interpretation. Healthcare, legal services, immigration, public services, NGOs, education, finance, tourism, and businesses are examples, not restrictions.

1.3 Goals and Objectives

The system aims to reduce language barriers; shorten the time needed to locate an appropriate interpreter; support reliable and traceable sessions; allow access for people without personal smartphones; protect sensitive information; provide operational staff with visibility into service delivery; and establish a foundation that can scale as interpreter capacity and institutional partnerships grow.

1.4 Intended Readers

Project sponsor and owner; business analyst; software engineers; UI/UX designers; quality-assurance testers; interpreters; supervisors/coordinators; platform administrators; security/privacy reviewers; and participating pilot institutions.

1.5 Definitions and Acronyms

SRS: Software Requirements Specification. API: Application Programming Interface. DRF: Django REST Framework. RBAC: Role-Based Access Control. PII: Personally Identifiable Information. WebRTC: technology for real-time audio/video communication. OTP: One-Time Password. SLA: Service-Level Agreement. Requester: person or authorized staff member initiating a request. Session: a coordinated interpretation interaction. Service user: the person who needs interpretation.

1.6 Reference Documents and Status

This SRS is based on the project concept, simple proposal, and project-structure diagrams discussed in October 2026. It is a draft baseline for review. Supported-language priorities, provider selection, detailed operating policies, legal/privacy requirements, service levels, and unresolved scope decisions require stakeholder approval.

2. Overall Description

2.1 Product Perspective

The product comprises a React.js/TypeScript web application, a Flutter Android application, a Django REST Framework backend, a PostgreSQL database, and an integration with a suitable real-time communication provider. Secure APIs connect clients. WebSockets or an equivalent event mechanism may deliver assignment and request-status updates. Audio/video sessions may use a managed WebRTC provider or another approved communication service.

2.2 Operating and Funding Model

The intended model is institution-funded: participating organizations contract for services and pay under separately agreed terms. End recipients such as patients, witnesses, clients, or members of the public are not directly charged by the platform in this release. A person should not be required to install an app if institution-assisted access through a phone, speakerphone, or approved browser link is suitable.

2.3 User Classes and Roles

Client/service user: submits or accesses an interpretation request, joins an authorized session, and may provide feedback. Interpreter: maintains languages, qualifications and availability; accepts or rejects assignments; interprets; and completes records. Supervisor/coordinator: monitors request queues, manually assigns or reassigns interpreters, resolves unknown-language cases, and reviews incomplete sessions. Platform administrator: manages platform-level accounts, roles, language catalog, configuration, interpreter verification status, system health, and audit review. Organization administration and organization staff-management interfaces are explicitly excluded.

2.4 Operating Environment

Web: supported current desktop and mobile browsers, finalized during testing. Android: Flutter application on an agreed minimum Android version. Server: supported Linux deployment environment, Django REST Framework, PostgreSQL, and approved third-party services. Production shall use HTTPS and protected environment configuration.

2.5 Design and Implementation Constraints

The project is bounded by the proposed development budget of ETB 90,000 and an estimated duration of up to three months, subject to a signed scope. Hosting, domain, SMS/OTP, telephony, email, audio/video usage, app-store charges, and other recurring provider costs are excluded unless specifically agreed. Access to national ID, hospital, court, immigration, or other institutional systems is not assumed and requires written authorization and technical agreements.

2.6 Assumptions and Dependencies

Pilot stakeholders will identify priority languages and provide an approved interpreter-verification process. Human interpreters and operational staffing will be arranged by the service operator. Internet access and compatible devices will be available for app/web sessions, with a documented phone-assisted fallback. Communication vendors will provide documented APIs and secure webhook mechanisms. Legal/privacy review will occur before live operation.

2.7 In-Scope Capabilities

Authentication and role permissions; interpreter profiles and availability; language catalog; immediate and scheduled requests; language/qualification matching; accept/reject and reassignment; secure session access; session status and completion records; coordinator queue and escalation; notifications when configured; platform-level configuration; audit logs; and operational reports.

2.8 Out of Scope

Organization administration, organization staff and permission management, organization self-service billing portals, advanced AI translation or speech interpretation, iOS app, automatic legal/medical decisions, automatic identity verification through national systems, guaranteed nationwide interpreter coverage, unapproved external integrations, default session recording, and third-party recurring charges. Full contracts, rate cards, invoices, payments, and organization billing dashboards are not committed features unless separately approved.

3. Functional Requirements

3.1 Authentication and Access Control

FR-001: The system shall authenticate users through a secure sign-in mechanism appropriate to their role. FR-002: Every protected API operation shall enforce server-side authorization. FR-003: The system shall support secure logout and token/session expiration. FR-004: Platform administrators shall be able to activate, suspend, or deactivate platform accounts. FR-005: Privileged accounts should use stronger authentication where feasible. FR-006: Unauthenticated users shall not access private profiles, requests, or session records. FR-007: The backend shall deny access by default and check resource ownership and role scope on every protected operation.

3.2 User and Interpreter Profiles

FR-008: Interpreter profiles shall include a unique identifier, display name, contact method, supported languages, language-pair direction where relevant, service categories, availability, verification state, and qualifications. FR-009: Interpreters shall update permitted profile fields and availability. FR-010: Qualification evidence shall be visible only to authorized roles. FR-011: Changes to verification status and qualifications shall be audited. FR-012: Interpreter states shall distinguish pending, verified, suspended, and rejected. FR-013: The system shall prevent suspended or unverified interpreters from receiving assignments that require verified status.

3.3 Language Catalog and Unknown-Language Handling

FR-014: The platform shall maintain a configurable catalog of supported languages and variants where useful. FR-015: Requesters shall select source and target languages when known. FR-016: The request form shall allow a requester to indicate that the language is unknown or cannot be selected. FR-017: Unknown-language requests shall be sent to a coordinator or approved human-identification workflow. FR-018: The system shall not represent an uncertain automatic language guess as confirmed, especially for high-stakes cases. FR-019: Administrators shall be able to mark languages active or inactive without deleting historical references.

3.4 Request Creation and Validation

FR-020: Authorized requesters shall create immediate or scheduled requests. FR-021: A request shall capture the minimum routing data: language(s), service category, urgency, requested communication mode, scheduling details if applicable, and limited reference/context if justified. FR-022: The system shall generate a unique, non-guessable request reference. FR-023: The requester shall see a confirmation and current status. FR-024: The system shall validate required fields, date/time values, and supported selections. FR-025: Invalid requests shall return understandable validation messages. FR-026: A requester or authorized coordinator shall be able to cancel a request before connection according to policy. FR-027: The system shall avoid collecting full medical histories, national-ID numbers, or unrelated sensitive details unless separately justified and approved.

3.5 Matching and Dispatch

FR-028: Matching shall consider language match, availability, verification status, qualifications, service category, schedule, and other approved eligibility constraints. FR-029: Exact language and qualification matches shall be preferred over less suitable candidates. FR-030: A coordinator shall be able to assign or reassign a request if matching fails or manual review is required. FR-031: Eligible interpreters shall receive assignment notifications through configured channels. FR-032: Interpreters shall accept or reject assignments, with a reason where appropriate. FR-033: On rejection or timeout, the system shall try another eligible interpreter or place the request in the coordinator queue. FR-034: The system shall prevent simultaneous acceptance of the same assignment by multiple interpreters. FR-035: Assignment attempts, outcomes, and reassignment reasons shall be recorded. FR-036: A request with no suitable interpreter shall remain visibly unresolved and must not be shown as successfully assigned.

3.6 Scheduled Requests

FR-037: Requesters shall be able to specify a future date and time where scheduling is enabled. FR-038: The system shall validate that the scheduled time is permitted by the service policy. FR-039: The request shall enter the scheduling queue and be assigned according to an approved lead-time policy. FR-040: The system shall support reminders when a notification channel is configured. FR-041: The system shall record schedule changes, cancellations, and assignment changes.

3.7 Session Creation and Communication

FR-042: A session shall be created only after assignment and authorization checks. FR-043: The system shall support the approved initial communication mode, such as audio or audio/video using a selected managed WebRTC provider. FR-044: Session credentials shall be short-lived, scoped, and revocable where supported. FR-045: Only authorized participants shall join a session. FR-046: The system shall display connection status and provide retry or coordinator escalation when connection fails. FR-047: Session start/end timestamps, duration, status, and outcome shall be recorded. FR-048: The platform may support a staff-assisted phone or speakerphone workflow. FR-049: Direct PSTN/telephony integration is not included unless a provider and scope are explicitly approved. FR-050: The application shall not claim a call is connected unless the communication provider or trusted session logic confirms it.

3.8 Session Completion and Feedback

FR-051: Interpreters shall mark sessions completed, interrupted, or not completed. FR-052: Completion records shall include session/request reference, outcome, start/end time, duration, interpreter, and a limited issue code or note. FR-053: Authorized service users or staff may provide feedback if enabled. FR-054: Feedback shall not be visible to unauthorized users. FR-055: Supervisors shall review disputed or incomplete sessions and record a resolution. FR-056: Status transitions and completion changes shall be auditable.

3.9 Supervisor / Coordinator Workspace

FR-057: Supervisors shall view pending, assigned, active, delayed, and unresolved requests within their authorized scope. FR-058: Supervisors shall filter by status, language, urgency, service category, and time. FR-059: Supervisors shall assign, reassign, cancel, or escalate requests, recording a reason when appropriate. FR-060: Supervisors shall review failed connections, missing interpreters, repeated rejections, and incomplete sessions. FR-061: Supervisors shall be able to mark an issue for follow-up and record its resolution. FR-062: Supervisors shall not have access to unrelated organizations' data or unnecessary sensitive case content.

3.10 Platform Administration

FR-063: Platform administrators shall manage platform-level account status and role assignment. FR-064: Platform administrators shall manage the supported-language catalog and global configuration. FR-065: Platform administrators shall review interpreter verification status and platform health. FR-066: Platform administrators shall access audit events and aggregate service statistics according to least privilege. FR-067: Administrative changes shall be recorded in audit logs. FR-068: Administrative role alone shall not grant unrestricted access to session content.

3.11 Notifications

FR-069: The system shall notify relevant users about request status, assignment, rejection/reassignment where applicable, cancellation, scheduled reminders, and completion. FR-070: Notification delivery state and retry attempts shall be stored when supported by the provider. FR-071: Notifications shall not include sensitive case details in SMS text, email subjects, or lock-screen messages. FR-072: Channels may include in-app/web, push, email, or SMS only when configured and funded. FR-073: If delivery fails, the system shall use an approved fallback or alert a coordinator.

3.12 Operational Reports

FR-074: Authorized roles shall view reports on request volume, language demand, response time, completion rate, cancellations, interpreter availability, and service failures. FR-075: Reports shall support date-range and approved category filters. FR-076: Aggregate reports shall minimize personal data. FR-077: Data exports shall be permission-controlled and audit logged. FR-078: Reports shall disclose limitations and shall not imply nationwide coverage without verified operational evidence.

3.13 Billing and Commercial Records — Scope Boundary

FR-079: The operating model is institution-funded, but organization administration and self-service organization billing are excluded from this release. FR-080: Basic session-usage records may be retained to support later reconciliation if approved. FR-081: Contracts, rate cards, invoices, payments, receipts, and organization billing dashboards are not committed features until separately scoped and approved. FR-082: Any manual platform-level billing ledger must be confirmed in writing before implementation.

3.14 Error Handling and Recovery

FR-083: Errors shall be understandable and shall not expose stack traces, secrets, or sensitive internal data. FR-084: Failed assignment, notification, session creation, and provider callback operations shall be logged with correlation identifiers. FR-085: The system should prevent duplicate requests caused by repeated submissions. FR-086: Transient provider failures shall be retried safely where appropriate. FR-087: When no interpreter is available, the system shall communicate that the request is awaiting escalation rather than falsely reporting success.

4. External Interface Requirements

4.1 User Interface

The user interface shall be responsive and readable on agreed desktop and mobile screen sizes. Critical actions—including accepting an assignment, joining a session, escalating a request, and ending a session—shall be visually distinct. Forms shall show plain-language validation errors. The initial interface language and localization scope must be confirmed; Amharic and other Ethiopian languages may be prioritized according to pilot needs.

4.2 Flutter Android Application

The Android app shall support the approved client/service-user flow: language selection or unknown-language option, service category/urgency, request submission, request status, authorized session joining, connection assistance, and feedback if included. It shall explain the next step after submission and provide an escalation route. Android minimum version, distribution method, offline behavior, and push provider must be confirmed.

4.3 React Web Platform

The web platform shall provide role-specific interfaces for interpreters, supervisors/coordinators, and platform administrators. Interpreter screens shall cover profile, availability, assignment inbox, session workspace, and completion records. Supervisor screens shall cover queues, manual dispatch, escalation, review, and operational reports. Platform administration screens shall cover global settings, language catalog, account/role management, and audit review. Organization administration is excluded.

4.4 Backend API

The backend shall expose documented, versioned REST APIs using Django REST Framework. Inputs shall be validated server-side, authorization enforced, and errors returned in a consistent structure. API documentation shall be delivered to the implementation and testing team. Sensitive fields shall be omitted from responses unless needed and authorized.

4.5 Database Interface

PostgreSQL shall be the primary relational database. Foreign keys and constraints shall protect data integrity. Frequently queried fields—including request status, language, assigned interpreter, and timestamps—shall be indexed where appropriate. Database migrations shall be versioned and reviewed.

4.6 Real-Time Events and Media

WebSockets or an equivalent mechanism may provide real-time assignment and request status updates. Audio/video shall use the selected managed WebRTC provider or another approved solution. Selection criteria shall include availability in Ethiopia, device/browser support, bandwidth use, privacy terms, cost, reliability, and support. Provider webhooks shall be verified and processed idempotently.

4.7 External Services

Optional integrations include push notifications, SMS/OTP, email, telephony, monitoring, and managed audio/video. Each requires approved credentials, secure secret storage, documented failure handling, and a defined cost owner. National ID, hospital, court, immigration, payment, and other institutional integrations are excluded unless expressly authorized and approved.

5. Data Requirements and Proposed Data Model

5.1 Data Principles

Collect only information required to route, secure, and evaluate the service. Separate operational metadata from sensitive case details. Use non-guessable identifiers, timestamps, foreign keys, validation constraints, and auditable status transitions. Avoid storing audio/video unless a separate approved requirement establishes purpose, lawful authority, notice/consent where applicable, access restrictions, retention, and deletion.

5.2 Core Entities

users: account identity, role, status, and authentication metadata. interpreter_profiles: languages, qualification/verification state, service categories, and availability. languages: codes, display names, variants, and active state. interpretation_requests: reference, languages, category, urgency, modality, schedule, status, requester reference, and minimum context. assignments: request, interpreter, assignment time, response, outcome, and reason. interpretation_sessions: session/provider reference, status, start/end time, duration, and outcome. session_participants: authorized participant and join/leave events. completion_reports: outcome, issue code, limited notes, and review state. notifications: channel, recipient, delivery state, retry count, and timestamps. audit_logs: actor, action, resource, time, result, and minimal security metadata. integration_events: provider callback reference, processing state, idempotency key, and retry details.

5.3 Relationships

A requester may create multiple requests. A request may have multiple assignment attempts but no more than one active accepted assignment at a time. A request may have session attempts according to the retry policy. A session may have authorized participants and a completion/review record. Audit records reference actors and resources without duplicating sensitive content.

5.4 Data Validation and Integrity

The system shall validate language identifiers, permitted status transitions, scheduling values, role permissions, assignment uniqueness, participant authorization, provider references, and duration calculations. Server-side validation is mandatory even where the frontend performs validation.

5.5 Retention, Archival, and Deletion

Retention periods shall be agreed with participating institutions and legal/privacy advisers before production. The system shall support controlled archival and deletion where permitted. Backups and logs shall have documented retention periods. Deletion shall not remove records that must lawfully be retained; any exception shall be documented.

5.6 Test Data and Seeding

Development/test environments shall use synthetic data wherever possible. Production seed data shall contain only approved language catalog entries and authorized platform accounts. Real case or personal data shall not be copied into development environments without formal approval and equivalent protection.

6. Non-Functional Requirements

6.1 Performance

NFR-001: Normal API operations should respond within 2 seconds at the 95th percentile under an agreed pilot load, excluding third-party provider latency. NFR-002: Request-status updates should appear within 5 seconds when network and notification services are available. NFR-003: Queues and reports shall use pagination and appropriate indexes. Concurrent-session and load targets must be finalized before performance testing.

6.2 Availability and Reliability

NFR-004: Pilot application availability should target 99% monthly, excluding planned maintenance and third-party outages, unless a different SLA is agreed. NFR-005: Critical workflows shall provide clear failure states and retry paths. NFR-006: The system shall not show a failed call or undelivered notification as successful. NFR-007: Backup and restoration procedures shall be documented and tested before live pilot.

6.3 Security

NFR-008: Production traffic shall use HTTPS/TLS. NFR-009: Locally managed passwords shall use modern adaptive password hashing. NFR-010: Secrets shall be stored outside source control. NFR-011: APIs shall validate input and enforce server-side authorization. NFR-012: Session join credentials shall be short-lived and scoped. NFR-013: Authentication and request endpoints shall have appropriate rate limits and abuse protection. NFR-014: Security-relevant actions shall be logged.

6.4 Privacy and Confidentiality

NFR-015: Personal and sensitive information shall be minimized. NFR-016: Recording shall be disabled by default. NFR-017: Sensitive records shall be accessible only to authorized roles with an operational need. NFR-018: Logs and notifications shall not expose unnecessary sensitive content. NFR-019: Legal/privacy review shall be completed before live use, including applicable Ethiopian laws, sector requirements, data sharing, and cross-border processing.

6.5 Usability and Accessibility

NFR-020: Primary request and join-session flows should be usable with minimal training. NFR-021: Interfaces shall use clear labels, consistent status terms, readable contrast, and accessible form feedback. NFR-022: Design shall account for low digital literacy and intermittent connectivity through simple flows and staff-assisted access. NFR-023: Language labels shall be understandable to intended users.

6.6 Maintainability

NFR-024: Frontend, backend, and data responsibilities shall be separated. NFR-025: Code shall follow documented conventions and version control. NFR-026: Environment-specific configuration shall be externalized. NFR-027: Automated tests shall cover authorization, request lifecycle, matching, and session rules. NFR-028: Deployment, backup, and recovery instructions shall be included in handover.

6.7 Scalability

NFR-029: The architecture should allow application workers and background tasks to scale independently. NFR-030: Queries and reports shall avoid unbounded retrieval. NFR-031: Nationwide service is a future objective dependent on interpreter capacity, connectivity, staffing, contracts, and infrastructure; it is not guaranteed by software delivery alone.

6.8 Compatibility and Observability

NFR-032: The system shall be tested on agreed browsers and Android devices. NFR-033: Logs shall include correlation IDs and timestamps. NFR-034: Monitoring shall surface API errors, queue backlog, provider failures, and notification delivery issues without exposing sensitive content.

7. Security, Safety, and Responsible Operation

7.1 Role-Based Access

The initial role model consists of client/service user, interpreter, supervisor/coordinator, and platform administrator. Organization administration is excluded. The backend shall check permissions and resource scope for each protected request and session operation. Platform administrator access shall be limited to legitimate platform operations.

7.2 High-Stakes Contexts

For healthcare, legal, immigration, and similar high-stakes contexts, the platform coordinates communication and does not replace professional judgment or qualified interpretation. Qualifications appropriate to the service category shall be checked. Urgent care or emergency response must not be delayed while searching for an interpreter; the interface and operating procedures shall state this.

7.3 Unknown Language and Escalation

Unknown-language requests shall be escalated to a coordinator or approved human-identification process. Any future automated detection shall communicate uncertainty and shall not be the sole basis for high-stakes decisions. Human escalation is required when confidence is low.

7.4 Recording and Consent

Audio/video recording shall be off by default. Any exception requires an approved purpose, lawful authority, appropriate notice/consent where applicable, access restrictions, retention schedule, and deletion process. The default design should not persist media after the session.

7.5 Audit and Incident Response

Audit logs shall cover relevant sign-in security events, role changes, interpreter verification, reassignment, sensitive-record access, exports, and administrative configuration. Incident procedures shall define reporting, containment, investigation, recovery, and stakeholder notification.

7.6 Abuse Prevention

The system should support rate limits, request throttling, account suspension, assignment timeouts, and coordinator escalation for abusive or repeated invalid requests. Safeguards must not prevent a legitimate urgent request from reaching a human coordinator.

8. Workflows and State Models

8.1 Immediate Request Workflow

1) Requester enters language and service details. 2) Backend validates and creates a request. 3) Matching identifies eligible interpreters. 4) Eligible interpreters are notified according to dispatch policy. 5) An interpreter accepts or rejects. 6) On acceptance, the system creates an authorized session. 7) Participants join through the approved channel. 8) Interpreter records the outcome and completion details. 9) Requester may provide feedback. 10) Authorized supervisors review operational status and service statistics.

8.2 Scheduled Request Workflow

1) Requester selects a future date/time. 2) The system validates scheduling constraints. 3) Request enters the scheduling queue. 4) An interpreter is assigned according to approved lead-time policy. 5) Reminders are sent if configured. 6) Confirmation, delays, reassignment, cancellation, and completion are recorded.

8.3 Assignment Retry Workflow

If an interpreter rejects, times out, becomes unavailable, or loses eligibility, the system records the outcome and attempts another eligible interpreter. Duplicate active assignments are prevented. After the retry limit or wait period, the request moves to the coordinator queue with a clear status.

8.4 Proposed Session States

created → searching → awaiting_acceptance → assigned → ready_to_join → active → completed. Alternative terminal/recovery states: cancelled, expired, no_interpreter_available, connection_failed, interrupted, and under_review. Only permitted transitions are allowed, and each transition records its time and actor/system source.

8.5 Proposed Request States

submitted, validating, matching, pending_coordinator, assigned, in_session, completed, cancelled, expired, and unresolved. Request and session status must remain consistent while preserving separate histories.

8.6 Phone or Speakerphone Access

Where a service user cannot install or use the Android app, authorized staff may initiate the request and connect the person through an institution-controlled phone or speakerphone. This is an operating procedure unless direct telephony integration is separately added. The request record should identify the initiating staff member and recording remains off by default.

9. System Architecture and Technology

9.1 Logical Architecture

Presentation: React.js/TypeScript web application and Flutter Android application. Application services: Django REST Framework APIs, authentication/authorization, request lifecycle, matching and dispatch, session orchestration, notifications, reporting, and audit logging. Data: PostgreSQL. Real-time updates: WebSockets or equivalent. Media: selected managed WebRTC provider or approved alternative. Background jobs may handle timeouts, reminders, retries, and notification delivery.

9.2 Backend Modules

Suggested modules: accounts/authentication; interpreter profiles and verification; language catalog; requests; matching and assignments; sessions and participants; notifications; supervisor queue; reports; audit logs; provider integrations; and platform settings. Organization administration and billing modules are excluded unless separately approved.

9.3 Web Modules

Sign-in; interpreter profile and availability; assignment inbox; session workspace; supervisor request queue; manual assignment and escalation; completion review; operational reports; platform settings and language catalog; platform account/role administration. No organization administration pages.

9.4 Android Modules

Request language selection; unknown-language option; service category and urgency; request submission; status tracking; authorized session joining; connection assistance; session history where approved; feedback where included; and notification handling if configured.

9.5 Integration Design

Provider-specific logic should be isolated behind service adapters. Webhooks shall be authenticated and processed idempotently. The system shall record provider references and failures without depending on callback order. Core request/session records should remain portable if a provider changes.

9.6 Environments and Deployment

Separate development, testing/staging, and production environments should be used. Production secrets/data shall not be reused in development. Database migrations shall be versioned. Deployment shall include health checks, logs, backup configuration, and rollback guidance.

9.7 Technology Baseline

React.js with TypeScript; Flutter for Android; Python/Django REST Framework; PostgreSQL; WebSockets or equivalent for real-time status; and managed WebRTC or approved audio/video service. Exact versions, hosting, notification provider, and media vendor require confirmation.

10. Testing and Acceptance

10.1 Test Levels

Unit tests shall cover business rules and validation. Integration tests shall cover database operations, APIs, matching, notifications, and provider callbacks. End-to-end tests shall cover request creation through completion. Security tests shall cover role restrictions, token expiry, cross-user access, input validation, and webhook verification. Usability tests shall include users with different levels of digital literacy.

10.2 Essential Test Scenarios

Test successful request and assignment; exact language match; qualification mismatch exclusion; rejection and retry; no interpreter available; unknown-language escalation; duplicate acceptance prevention; unauthorized session join denial; session interruption/completion; notification failure; request cancellation; scheduled reminders; audit history; report date filtering; Android connection failure; and access denial for restricted records.

10.3 Acceptance Criteria

The release may be accepted when: (a) approved role flows work end-to-end; (b) requests can be created, matched, assigned, joined, and completed; (c) unknown-language and no-interpreter cases escalate safely; (d) only authorized participants can join sessions; (e) supervisor queues and reports show consistent records; (f) critical authorization/security tests pass; (g) deployment, backup, API, and handover documentation is delivered; and (h) limitations and recurring third-party costs are documented.

10.4 Defect Severity

Critical: unauthorized sensitive-data access, session hijacking, data loss, or unsafe false-success state. High: core request, matching, or session workflow blocked. Medium: significant usability/reporting issue with workaround. Low: cosmetic or minor text defect. Critical and high defects must be resolved or explicitly accepted in writing before pilot launch.

10.5 Pilot Readiness

Before live use, stakeholders must approve supported languages, interpreter qualification checks, escalation staffing, data retention, privacy notice/consent, incident handling, communication provider, operating hours, support contact, and emergency fallback. Software delivery alone does not establish interpreter supply or operational readiness.

11. Implementation Plan and Deliverables

11.1 Proposed Phases

Phase 1: requirements validation, workflow confirmation, and UI planning. Phase 2: Django backend, PostgreSQL schema, authentication, and APIs. Phase 3: React web modules for interpreters, supervisors, and platform administrators. Phase 4: Flutter Android client workflows. Phase 5: real-time session/provider integration. Phase 6: integration testing, security checks, bug fixing, deployment, documentation, and handover.

11.2 Deliverables

Approved SRS baseline; application source code; database schema and migrations; REST API documentation; React web application; Flutter Android application source; configured backend; selected communication integration as agreed; test cases and test results; deployment/configuration guide; operational quick-start notes; and handover session.

11.3 Schedule

The proposal estimates up to three months. Actual duration depends on stakeholder decisions, provider selection, access to test devices, interpreter workflow validation, feedback turnaround, and scope changes. A milestone schedule should be agreed before development.

11.4 Change Control

A change request shall describe the requirement, reason, affected modules, security/privacy implications, cost, schedule, and acceptance tests. Expanding scope to organization administration, billing portals, direct telephony, recording, AI translation, iOS, or institutional integrations requires written approval and may affect price and delivery.

11.5 Handover and Support

Handover shall include source repositories, an environment-variable inventory without secret values, setup instructions, migration commands, deployment notes, backup/restore guidance, API documentation, and known limitations. Hosting, third-party usage, interpreter operations, and long-term support are separate unless agreed.

12. Risks, Dependencies, and Open Decisions

12.1 Risks

Limited qualified interpreters for some languages; unreliable connectivity; provider cost or outages; unclear sector privacy obligations; poor language/qualification matching; users without smartphones; abusive requests; delayed decisions; and scope expansion beyond budget.

12.2 Mitigation

Pilot with selected languages and institutions; retain coordinator escalation; document emergency fallback; verify interpreter qualifications; evaluate providers for cost/connectivity; minimize sensitive data; test authorization and recovery; agree service hours and staffing; and enforce written change control.

12.3 Decisions Required Before Development

Confirm initial languages/variants; service categories; audio-only versus audio/video; communication provider; direct phone integration versus staff-assisted calls; Android minimum version; supported browsers; interface languages; interpreter verification process; assignment timeout/retry rules; data retention; feedback/dispute handling; whether any billing ledger is included; hosting owner; service hours; and pilot SLA.

12.4 External Dependencies

Internet and mobile data; Android devices and browsers; hosting/database infrastructure; managed audio/video if selected; push/SMS/email providers if enabled; participating institutions and operational staff; verified interpreters; and legal/privacy review. National ID or institutional data access requires formal authorization.

12.5 Budget Boundary

The proposal reference is ETB 90,000 for development and up to three months. The amount does not automatically include hosting, domain, SMS/OTP, telephony, email, audio/video usage, app-store fees, legal advice, interpreter recruitment, or ongoing operations. Items not explicitly included must be confirmed before commitment.

13. Appendices

Appendix A. Suggested API Domains

Authentication/accounts: /api/v1/auth and /api/v1/users. Languages: /api/v1/languages. Interpreter profiles/availability: /api/v1/interpreters and /api/v1/availability. Requests/assignments: /api/v1/requests and /api/v1/assignments. Sessions/participants: /api/v1/sessions and /api/v1/participants. Completion/feedback: /api/v1/completions and /api/v1/feedback. Supervisor queue/assignments: /api/v1/supervisor/queue and /api/v1/supervisor/assignments. Notifications: /api/v1/notifications. Reports: /api/v1/reports. Platform settings/audit: /api/v1/platform-settings and /api/v1/audit-logs. These are proposed domains, not a finalized API contract.

Appendix B. Initial Role Permission Summary

Client/service user: create/view own permitted requests, join authorized sessions, and submit feedback. Interpreter: view assigned requests, accept/reject, update availability, join assigned sessions, and submit completion records. Supervisor/coordinator: view authorized queues, assign/reassign/escalate, review completion/issues, and view operational reports. Platform administrator: manage platform accounts/roles, language catalog, global settings, and audit review. No organization-administrator role is included.

Appendix C. Requirement Priority

P1 — essential for pilot: authentication/authorization, language catalog, request lifecycle, eligibility and dispatch, session authorization, completion records, coordinator queue, audit logs, unknown-language escalation, and core reports. P2 — desirable if schedule allows: scheduled requests, reminders, feedback, enhanced report filters, notification fallback, and expanded localization. P3 — future/separately approved: advanced AI language identification/translation, iOS, direct telephony, recording, organization administration, organization billing portal, and institutional integrations.

Appendix D. Approval Record

Project owner: _____ Date: _____. Technical lead: _____ Date: _____. Pilot stakeholder: _____ Date: _____. Approved version: _____. Approval confirms the requirements baseline; later changes remain subject to change control.